

Первое знакомство

```
package main

import "fmt"

func main() {
    add := func(a, b int) int {
        return a + b
    }

    result := add(3, 4)
    fmt.Println(result) // 7
}
```

Передача как callback аргумент

```
package main

import "fmt"
import "slices"

func main() {
    words := []string{"aa", "aaa", "a"}
    slices.SortFunc(words, func(a, b string) int {
        return len(a) - len(b)
    })
    fmt.Println(words) // [a aa aaa]
}

// func Slice(slice interface{}, less func(i, j int) bool)
```

Замыкания

```
package main
import "fmt"
func get_generator() func() int {
    i := 0 // 'i' will copy by reference
    return func() int {
        i++
        return i
    }
}
func main() {
    a := get_generator()
    fmt.Println(a()) // 1
    fmt.Println(a()) // 2

    b := get_generator()
    fmt.Println(b()) // 1
}
```

Замыкания - передача по копии или ссылке?

```
package main
import "fmt"
func call(f func(bool) int) { f(true) }
func main() {
    i := 0
    // 'i' will copy by reference
    f := func(inc bool) int {
        if inc { i++ }
        return i
    }
    g := func(val int) func(bool) int {
        return func(inc bool) int {
            if inc { val++ }
            return val
        }
    }(i)
```

```
    call(f)
    fmt.Println(i, f(false), g(false)) // 1, 1, 0
    call(g)
    fmt.Println(i, f(false), g(false)) // 1, 1, 1
}
```

Замыкания - изучение убегания на кучу

```
package main

func EscapingClosure() func() int {
    x := 42
    return func() int {
        x++
        return x
    }
}

func NoEscapingClosure() int {
    y := 42
    return func() int {
        return y
    }()
}
```

```
$ go mod init example/escape
$ go test -gcflags="-m" -bench .
# example/escape
./main.go:6:6: can inline EscapingClosure
./main.go:8:10: can inline EscapingClosure.func1
./main.go:16:10: can inline NoEscapingClosure.func1
./main.go:14:6: can inline NoEscapingClosure
./main.go:18:4: inlining call to
                    NoEscapingClosure.func1
./main.go:7:3: moved to heap: x
./main.go:8:10: func literal escapes to heap
?          example/escape      [no test files]
```

Ошибки с горутинами

```
package main
import (
    "fmt"
    "sync"
)
func main() {
    var wg sync.WaitGroup
    for i := 0; i < 15; i++ {
        wg.Add(1)
        go func() {
            fmt.Print(i)
            wg.Done()
        }()
    }
    wg.Wait()
}
```

```
$ go run ./main.go
146527493121008111113%
$ go run ./main.go
14678910111213324501%
$ go run ./main.go
14012345678912131011%
```